

munx

Language Specification

A systems language with first-class concurrency and I/O

Version 0.1 — derived from the reference front-end in this repository

Source grammar: `sample/**/*.mx`

This document describes the lexical structure, types, expressions, statements, modules, concurrency primitives, and abstract syntax of **munx**. The normative grammar is the behaviour of the reference lexer and parser (`include/lexer.hpp`, `include/parser.hpp`).

Contents

1. Introduction
2. Lexical structure
3. Types
4. Expressions
5. Statements and declarations
6. Modules and packages
7. Concurrency and I/O
8. Memory management
9. Standard builtins (prelude)
10. Abstract syntax tree
11. Operator precedence
12. Grammar sketch
13. Example programs
14. Implementation notes

1. Introduction

munx is a systems-oriented programming language. Programs are stored in `.mx` source files. The language emphasises explicit concurrency (threads, locks, pipes, join) and I/O (sockets, files, terminals) as ordinary values rather than hidden runtime magic.

The reference implementation in this repository is a front-end only: it lexes and parses source into an AST. Bytecode generation and VM execution are out of scope for the front-end, but opcodes are sketched in `include/Opcode.hpp`.

1.1 Design principles

- Sample-driven grammar — accepted syntax is whatever `sample/` uses.
- Structural keywords are few; most builtins remain ordinary identifiers so they can be rebound (e.g. `print = fix(...)`).
- Semicolons are optional statement terminators.
- Blocks are brace-delimited; indentation is not significant.
- Only enums may be matched with `match`.
- Casts apply only between primitive types.

2. Lexical structure

2.1 Encoding and whitespace

Source is a sequence of Unicode characters treated as bytes by the reference lexer. Whitespace (space, tab, CR, LF) separates tokens and is otherwise insignificant.

2.2 Comments

- Line comments: `//` to end of line.
- Block comments: `/* ... */`, non-nesting. An unclosed block comment is a compile error.

2.3 Identifiers

An identifier starts with a letter or `_` and continues with letters, digits, or `_`. Identifiers are case-sensitive. The single underscore `_` is a discard target in destructuring.

2.4 Keywords

The following words are tokenized as `KEYWORD`. Elsewhere they may still appear where an identifier is expected (names, members, enum members), except that `load_package / load_packages` are rejected outside the import header.

```
package load_package load_packages
func lambda return if else loop break
match case enum object tuple
alloc delete free
lock acquire release join
monitor trap cast fail
```

2.5 Literals

Kind	Form	Notes
Integer	<code>0, 42, 007</code>	Signed 64-bit. Overflow is a compile error.
Float	<code>3.14, 0.5</code>	Digits required on both sides of <code>..</code>
String	<code>"..."</code>	Escapes: <code>\n \t \a \\ \" \'</code> .
Character	<code>'a', '\n'</code>	Exactly one character (after escapes).

Kind	Form	Notes
Boolean	true, false	Literals, not keywords.
Null	null	Absence of a value (e.g. closed stream).
Regex	r"..."	Raw pattern; no escape processing; cannot contain ".".

2.6 Operators and punctuation

```
( ) { } [ ] , : ; . ::
= += == != < > <= >=
+ - * / % ! ~ && ||
=> -> <-
```

-> inserts into a named pipe; <- extracts from a named pipe (blocking). => introduces a lambda body. The tokens &, |, and ^ are recognised by the lexer but have no parser production (bitwise binary ops are currently unsupported).

3. Types

Types appear in parameter lists, return annotations, object fields, cast targets, trap parameters, and typed array literals. Expression nodes in the AST are untyped; inference is left to later stages.

3.1 Type forms

Form	Syntax	Meaning
Primitive	int, file, ...	Built-in scalar or I/O type.
Named	Person	User-defined enum or object name.
Array	[T]	Homogeneous dynamic array of T.
Tuple	tuple[T1, T2, ...]	Fixed product type; may be empty: tuple[].

3.2 Primitive types

Name	Role
int	Signed 64-bit integer.
float	Floating-point number.
bool	Boolean; used in conditions.
string	UTF-8 text (implementation-defined encoding).
character	Single character value.
void	No value; only as a return type.
socket	Network socket from <code>open(io_type::socket, ...)</code> .
file	Filesystem handle from <code>open(io_type::file, ...)</code> .
term	Terminal/TTY handle from <code>open(io_type::tty, ...)</code> .
exception	Error value delivered to a trap handler.

The three I/O object types are selected by the first argument to `open`:

open kind	Result type
io_type::socket	socket
io_type::file	file
io_type::tty	term

Enum members such as `io_type::file` are ordinary enum-access expressions. Only type *positions* (parameters, fields, casts, traps) resolve the name `file` to the primitive.

3.3 Casts

Syntax: `cast[T](expr)`. Casts are defined only for primitive types. Converting compound or user types must be implemented manually.

```
i = cast[int]("134")
s = cast[string](i)           // "134"
cast[string]([int][1,2,3])    // "[1,2,3]"
```

4. Expressions

4.1 Primary expressions

- Literals (see §2.5)
- Identifiers and Enum: `:Member` access
- Grouped expressions `(e)`
- Array literals `[e, ...]` and typed arrays `[T][e, ...]`
- Tuple literals `{e, ...}` (including `{}`)
- Casts, `alloc`, `delete / free`, `lambdas`

4.2 Postfix

- Member access: `obj.field`
- Call: `f(a, b)`
- Index: `items[i]` (single index expression)

4.3 Unary operators

```
!expr    // logical not
~expr     // bitwise not
-expr     // negation
<- name  // blocking pipe extract
```

4.4 Binary operators

```
* / %      // multiplicative
+ -        // additive
< > <= >=   // relational
== !=      // equality
&&         // logical and
||         // logical or
expr -> name // pipe insert (right side is a pipe name)
```

4.5 Lambdas

```
f = lambda (x: int): int => {
    return x * 2
}

// immediately invoked
result = lambda (x: int): int => { return x + 1 }(41)
```

A lambda always has a parameter list, a return type, `=>`, and a block body. Unlike `func`, the arrow is required.

5. Statements and declarations

5.1 Assignment and destructuring

An assignment target is always a plain name (or `_`). Member and index expressions cannot appear on the left-hand side.

```
x = 1
x += 2
a, b = divide(9, 2)
{data, addr} = read(server, 1024)
{resp, _} = read(client, 1024)
```

5.2 Control flow

```
if cond {
  ...
} else if other {
  ...
} else {
  ...
}

loop cond { ... }    // conditional
loop { ... break }   // infinite until break

match value {
  case Color::Red { ... }
  case Color::Blue { ... }
}
```

match scrutinees must be enums. Each case names `Enum::Member` and a block. Zero or more cases are allowed.

5.3 Functions

```
func double(param: int): int {
  return param * 2
}

func divide(a: int, b: int): tuple[int, int] {
  return {a / b, a % b}
}
```

Every function has a return type annotation. Parameters are `name: type`. Nested function, enum, and object declarations are permitted inside blocks.

5.4 Enums and objects

```
enum Gender { Male, Female }

object Person {
  name: string,
  age: int,
  gender: Gender
}

person = Person("Ada", 36, Gender::Female)
```

Object construction is an ordinary call whose callee is the type name. Trailing commas in enum or object member lists are not allowed.

5.5 Exception handling

```
monitor {
  risky()
} trap(err: exception) {
  println(err)
}
```

A monitor block is always followed by a `trap(name: type)` handler block. `fail(message)` aborts with an error.

5.6 Blocks and layout

A bare `{ ... }` is a block statement. Semicolons after statements are optional. A tuple literal cannot begin a statement, because `{` opens a block — bind the tuple to a name first.

6. Modules and packages

Every source file begins with a package declaration, optionally followed by an import header:

```
package chatrelay
load_package types
load_packages {logger, server}

// first non-import statement ends the header
x = 1
```

- `load_package name` — import one package.
- `load_packages {a, b, ...}` — import several; the list must be non-empty and must not end with a trailing comma.
- Both forms may repeat and interleave, but only before the first statement.
- An import after a declaration, or inside any block/function/lambda/branch, is a compile error.
- The package declaration itself must be the first token of the file.

7. Concurrency and I/O

7.1 Threads and join

```
t = thread(lambda (n: int): void => {
  print(n)
}, {1})
join [t]
join [t1, t2, t3] // names only; may be empty: join []
```

7.2 Locks

```
lock counter_lock
acquire counter_lock
counter += 1
release counter_lock
```

7.3 Pipes

Pipes are named channels. Creation is an ordinary call; insertion and extraction use dedicated operators. The pipe operand of `->` / `<-` must be a bare identifier (not an expression).

```
out_pipe = pipe("id", out)
42 -> out_pipe
value = <- out_pipe // blocks until a value is available
```

7.4 Sockets, files, and terminals

```

server = open(io_type::socket, af_type::inet, sock_type::stream)
bind(server, "127.0.0.1", 8000)
listen(server, 128)
client, addr = accept(server)
data = read(client, 1024)
write(client, "ok")
close(client)

f = open(io_type::file, "/tmp/log.txt", out)
print = fix(open(io_type::tty, out))

```

`fix(expr)` binds a reference and prevents reassignment — used for global terminal handles such as `print`, `println`, and `readln`.

8. Memory management

```

buff = alloc [100] [1, 2, 3] // capacity, then initial values
empty = alloc [64] []
delete buff
free empty // alias of delete

```

Both the capacity brackets and the initializer brackets are required. `delete` / `free` take a buffer name.

9. Standard builtins (prelude)

Builtins are ordinary identifiers resolved by the runtime, not structural keywords. The samples establish the following surface (non-exhaustive):

Area	Names
I/O	<code>open</code> <code>bind</code> <code>listen</code> <code>accept</code> <code>connect</code> <code>read</code> <code>write</code> <code>close</code>
Terminal	<code>print</code> <code>println</code> <code>readln</code> <code>fix</code> <code>in</code> <code>out</code>
Concurrency	<code>thread</code> <code>pipe</code> <code>queue</code> <code>push</code> <code>pop</code> <code>sleep</code> <code>process</code>
Strings / arrays	<code>concat</code> <code>len</code> <code>split</code> <code>trim</code> <code>append</code> <code>remove_at</code> <code>has_substring</code> <code>has_substring_regex</code>
Control	<code>argv</code> (implicit argument array)

10. Abstract syntax tree

Nodes use an explicit discriminator plus a `std::variant` payload (`include/ast.hpp`). Helpers `make_expr_ptr`, `make_stmt_ptr`, `as<T>`, and `as_stmt<T>` keep the discriminator aligned with `value.index()`.

10.1 Root

```

struct program {
    string package_name;
    vector<load_package_stmt> imports;
    vector<unique_ptr<stmt_node>> statements;
};

```

10.2 Expression kinds (expr_type)

`IntLiteral` · `FloatLiteral` · `StringLiteral` · `CharLiteral` · `BoolLiteral` · `NullLiteral` · `RegexLiteral` · `Identifier` · `Binary` · `Unary` · `Call` · `Member` · `EnumAccess` · `Index` · `ArrayLiteral` · `TypedArrayLiteral` · `TupleLiteral` · `PipeInsert` · `PipeExtract` · `Cast` · `Alloc` · `Free` · `Lambda`

10.3 Statement kinds (stmt_type)

Assignment · Expr · Return · Break · Block · If · Loop · Match · FuncDecl · EnumDecl · ObjectDecl · Monitor · Lock · Acquire · Release · LoadPackage

10.4 Type kinds (type_kind)

Primitive · Named · Array · Tuple — with `primitive_kind` values `Int`, `Float`, `Bool`, `String`, `Character`, `Void`, `Socket`, `File`, `Term`, `Exception`.

11. Operator precedence

From tightest to loosest binding:

Level	Operators	Associativity
1 Postfix	<code>.</code> <code>()</code> <code>[]</code>	Left
2 Unary	<code>!</code> <code>~</code> <code>-</code> <code><-</code>	Right
3 Multiplicative	<code>*</code> <code>/</code> <code>%</code>	Left
4 Additive	<code>+</code> <code>-</code>	Left
5 Relational	<code><</code> <code>></code> <code><=</code> <code>>=</code>	Left
6 Equality	<code>==</code> <code>!=</code>	Left
7 Logical and	<code>&&</code>	Left
8 Logical or	<code> </code>	Left
9 Pipe insert	<code>-></code>	None (suffix)
10 Assignment	<code>=</code> <code>+=</code>	Statement

12. Grammar sketch

Informal EBNF reflecting the reference parser. `name` is an identifier or keyword used as a name.

<code>program</code>	<code>= 'package' name { import } { statement } ;</code>
<code>import</code>	<code>= 'load_package' name</code> <code> 'load_packages' '{' name { ',' name } '}' ;</code>
<code>statement</code>	<code>= func_decl enum_decl object_decl</code> <code> if_stmt loop_stmt match_stmt monitor_stmt</code> <code> 'lock' name 'acquire' name 'release' name</code> <code> 'return' [expr] 'break' 'join' '[' [name { ',' name }] ']'</code> <code> assignment expr block ;</code>
<code>assignment</code>	<code>= targets ('=' '+= ') expr ;</code>
<code>targets</code>	<code>= name { ',' name }</code> <code> '{' name { ',' name } '}' ;</code>
<code>type</code>	<code>= 'tuple' '[' [type { ',' type }] ']'</code> <code> '[' type ']'</code> <code> name ;</code>
<code>expr</code>	<code>= pipe ; /* see precedence table */</code>
<code>lambda</code>	<code>= 'lambda' '(' params ')' ':' type '=>' block ;</code>
<code>block</code>	<code>= '{' { statement } '}' ;</code>

13. Example programs

13.1 Minimal package

```
package hello
println = fix(open(io_type::tty, out))
println("hello, munx")
```

13.2 Function, match, and pipes

```
package demo
load_package types

enum Color { Red, Green, Blue }

func paint(c: Color): void {
  match c {
    case Color::Red { print("red") }
    case Color::Green { print("green") }
    case Color::Blue { print("blue") }
  }
}

p = pipe("ready", out)
1 -> p
flag = <- p
```

13.3 Larger project

The reference tree includes `sample/chatrelay/`, a multi-file concurrent TCP chat relay (roster with locks, async file logger over a pipe, per-client threads, line protocol). Parse it with:

```
./munx --files sample/chatrelay/*.mx
```

14. Implementation notes

- Build the front-end with `clang++ -std=c++20 -Iinclude src/main.cpp -o munx`.
- CLI: `./munx file.mx, --tokens, --files f0 f1 ....`
- Happy-path and error-path corpora live under `sample/valid/` and `sample/invalid/`.
- Expression nodes carry no type information — infer during analysis or codegen.
- `join [a,b]` lowers to a call; I/O and threading builtins are ordinary calls recognised by name.
- Suggested opcodes: `PUSH POP ADD CALL OPEN READ ...` in `Opcode.hpp`.

— End of specification —